

직관이 아닌 데이터로 정책을 설계하다.

공공데이터 기반 정책 수립을 위한

고령자 교통사고 위험 예측 및 서비스 구현

1조

박승주 · 노한별 · 이승기 · 전민서 · 이현승



목차

CONTENTS

01 분석 개요
목표와 문제의식 소개

02 분석 방법
데이터 수집, 전처리, 분석

03 분석 결과
주요 결과 및 인사이트 제시

04 서비스 구현
웹 서비스 및 모델 적용 사례

05 확장성
타 계층 적용 및 활용 방안

06 기대효과
예산, 정책 등 효과적 활용 전망

07 질의응답
Q & A





1. 분석 개요



“지금, 세금은 어디로 가고 있을까?”



출처 : 국토교통부_ 2024년 예산개요

③ (자동차 및 교통정책) 안전사고 예방 등을 위한 교통사고예방지원(30억원), '27년 완전 자율주행 상용화를 위한 테스트베드 환경 고도화 등 자율차 상용화 기반 마련 추진(429억원)



교통사고 예방 지원 예산 30억



현황 분석

실제 특정구 예산 편성 현황



예산 편성 사유 확인 불가

대상 선정

"고령자" 선정 이유



급속한 인구 구조 변화
**65세 이상 고령자
약 20%**

치명적인 사고 피해 계층
보행 사망율 67%

제도적 보호의 사각지대
**어린이 보호구역 11억원
고령자 보호구역 443만원**



프로젝트 개요

Project Overview



| 주요내용

고령자 교통사고 데이터를 분석하여 고위험군을 시각화하고,
지역별 사고 유형과 발생 위험도를 예측하는 웹 기반 예측 서비스 구현

| 진행 목적

데이터 기반 정책 판단 근거 제공
예산의 효율적 분배 및 교통안전 사각지대 해소

| 대상

정책 입안과 행정에 활용 가능한 실무자/지자체 관계자
지역 사회 교통안전 문제에 문제의식을 가진 시민

| 개발 기간

2025.05.26(월) ~ 2025.06.09(월)

| 개발 범위

전국 고령자 교통사고 데이터 분석
고위험 지역 시각화
사고 유형별 위험도 예측
웹 서비스 시제품 개발 및 시연





2. 분석 방법



데이터 소개

사용한 데이터 소개



고령자 교통사고 예측을 위한 데이터 수집 및 통합



한국도로교통공단
TAAS

연령별·지역별
교통사고 발생 정보



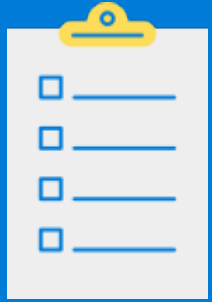
KOSIS KOSIS
국가통계포털

고령자 인구 비율,
시군구별 인구 구조 통계



공공데이터 포털

도로 형태, 주소기반 위치
데이터 등
행정구역별 부가 정보



데이터 전처리 흐름도



가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.

| 검정 방법

ANOVA

| 결과 요약

- ✓ 사고 수는 연도별 유의미한 차이를 보였음
- ✓ 2030년까지 지속적 증가가 예측됨
- ✓ 연도와 피해 유형간의 연관성도 파악하기 위해 SARIMAX 사용

가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.

```
df_total = pd.read_csv('../data/preprocessed_data/시도_시군구별_보행자_사고_사고유형_전처리ver.csv')
# df_total.head()
HTML(df_total.head().to_html())
```

	연도	시군구	법정동코드	사고내용	사망자수	중상자수	경상자수	부상신고자수	사고유형	도로형태	피해자 상해정도
0	2020	서울특별시 종로구 창성동	1111010500	중상사고	0	1	0	0	차대사람 - 차도통행중	교차로 - 교차로부근	중상
1	2020	서울특별시 종로구 창성동	1111010500	중상사고	0	1	0	0	차대사람 - 차도통행중	교차로 - 교차로안	중상
2	2020	서울특별시 종로구 통인동	1111010800	경상사고	0	0	1	0	차대사람 - 기타	단일로 - 기타	경상
3	2020	서울특별시 종로구 누상동	1111010900	경상사고	0	0	1	0	차대사람 - 차도통행중	교차로 - 교차로안	경상
4	2020	서울특별시 종로구 누상동	1111010900	경상사고	0	0	1	0	차대사람 - 기타	단일로 - 기타	경상

가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.

```
# ANOVA 검정
groups = [df_total[df_total['연도'] == year]['경상자수'] for year in [2020, 2021, 2022, 2023, 2024]]
f_stat, p_val = f_oneway(*groups)
print(f"F-통계량: {f_stat:.4f}, p-value: {p_val:.4f}")

# =====
# 연도 간 차이가 무작위(우연)보다 약 4.6배 큼
# p-value < 0.05이므로 귀무가설 기각
# 연도에 따라 고령자 사고 피해 수에 유의미한 차이 존재
# =====
```

]

Python

. F-통계량: 4.5946, p-value: 0.0010

가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.

```
# 귀무가설(H0): 연도와 피해 유형간의 연관성이 없다.
# 대립가설(H1): 연도와 피해 유형간의 연관성이 있다.

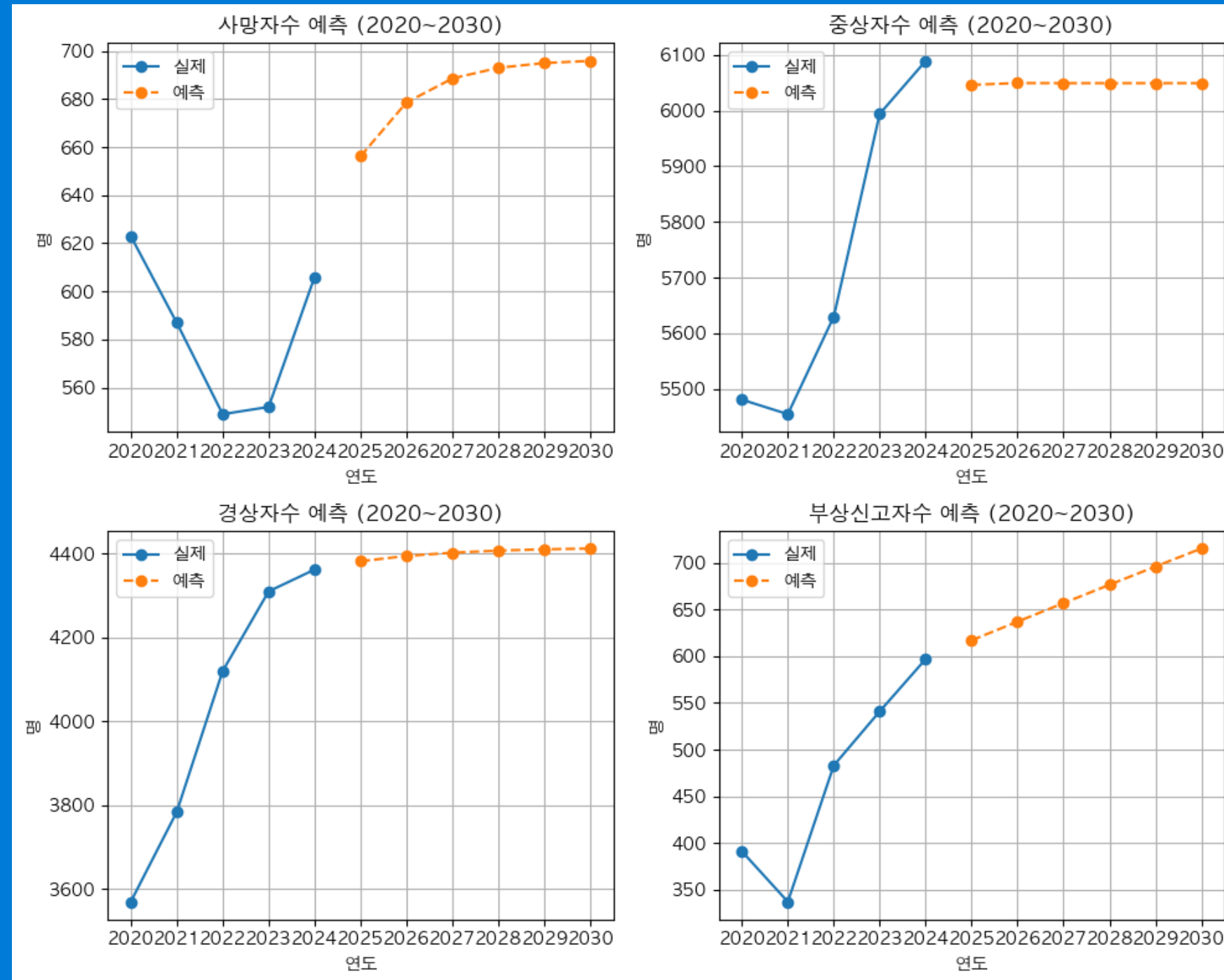
# 연도 vs 피해유형 피벗 테이블 생성
df_cross = pd.pivot_table(
    df_total,
    values='사망자수',
    index='연도',
    aggfunc='sum'
).join([
    df_total.pivot_table(values='중상자수', index='연도', aggfunc='sum'),
    df_total.pivot_table(values='경상자수', index='연도', aggfunc='sum'),
    df_total.pivot_table(values='부상신고자수', index='연도', aggfunc='sum')
])

chi2, p_val, dof, expected = chi2_contingency(df_cross)
print(f"Chi-square: {chi2:.4f}, p-value: {p_val:.15f}")

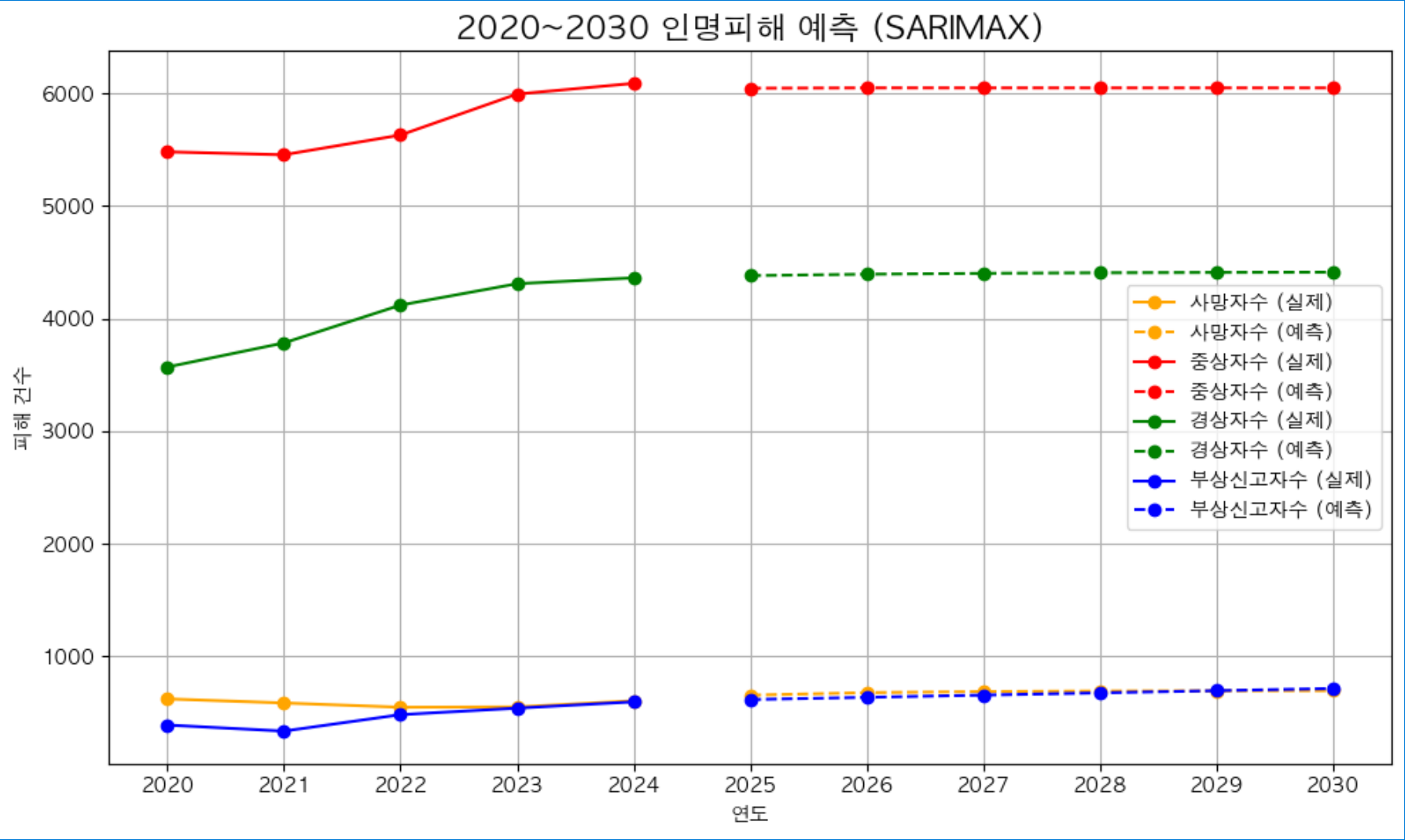
# =====
# 2020년과 2024년 사이, 고령자 교통사고에서의 피해유형 분포가 변했을까?
# Chi-square: 94.2524 -> 연도와 피해 유형간의 연관성이 매우 크다
# p-value < 0.05이므로 귀무가설 기각
# 피해 유형별로 연도에 따른 분포 차이가 유의미하게 존재한다.
# 피해의 종류(사망/중상/경상 등) 측면에서도 연도에 따라 달라진다.
# =====
```

Chi-square: 94.2524, p-value: 0.0000000000000007

가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.



가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.



가설 1. 연도가 지남에 따라 고령자 교통사고는 증가하고 있다.

| 검정 방법

ANOVA

| 결과 요약

- ✓ 사고 수는 연도별 유의미한 차이를 보였음
- ✓ 2030년까지 지속적 증가가 예측됨
- ✓ 연도와 피해 유형간의 연관성도 파악하기 위해 SARIMAX 사용

가설 2. 고령자 비율이 높은 지역일수록 사고가 많을 것이다.

| 검정 방법

피어슨/스피어만 상관계수 + OLS 회귀

| 결과 요약

- ✓ 오히려 음의 상관관계
- ✓ 고령자 비율이 1%p 높을수록 사고 피해자 수 약 13명 감소
→ 인구밀도·교통량 등 외부 요인이 더 영향

가설 2. 고령자 비율이 높은 지역일수록 사고가 많을 것이다.

| 검정 방법

피어슨/스피어만 상관계수 + OLS 회귀

| 결과 요약

- ✓ 오히려 음의 상관관계
- ✓ 고령자 비율이 1%p 높을수록 사고 피해자 수 약 13명 감소
→ 인구밀도·교통량 등 외부 요인이 더 영향

```
corr, p = pearsonr(df_merged['고령자 비율 (%)'], df_merged['피해자수'])
print(f'피어슨 상관관계수: {corr:.3f}, p-value: {p:.30f}')
```

피어슨 상관계수: -0.584 , p-value: $0.00000000000000000000210816590$

```
corr, p = spearmanr(df_merged['고령자 비율 (%)'], df_merged['피해자수'])
print(f'스피어만 상관계수: {corr:.3f}, p-value: {p:.30f}')
```

[illegible]

가설 2. 고령자 비율이 높은 지역일수록 사고가 많을 것이다.

검정 방법

피어슨/스피어만 상관계수 + OLS 회귀

결과 요약

- ✓ 오히려 음의 상관관계
- ✓ 고령자 비율이 1%p 높을수록 사고 피해자 수 약 13명 감소
→ 인구밀도·교통량 등 외부 요인이 더 영향

=====
고령자 비율이 1% 증가할 때, 피해자 수는 평균적으로 약 12.88명 감소
p-value는 0.000이므로, 회귀계수가 유의미
귀무가설 기각
결정계수(R²)=0.341 -> 고령자 비율만으로 고령자 피해자 수의 약 34.1%를 설명 가능. 나머지는 고령자 절대 수, 차량 수, 인구 밀도 등에 의해 설명 가능.
=====

OLS Regression Results						
=====						
Dep. Variable:	피해자수		R-squared:	0.341		
Model:	OLS		Adj. R-squared:	0.338		
Method:	Least Squares		F-statistic:	117.9		
Date:	Sun, 01 Jun 2025		Prob (F-statistic):	2.11e-22		
Time:	22:59:19		Log-Likelihood:	-1488.6		
No. Observations:	230		AIC:	2981.		
Df Residuals:	228		BIC:	2988.		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	540.9925	30.362	17.818	0.000	481.167	600.818
고령자 비율 (%)	-12.8793	1.186	-10.856	0.000	-15.217	-10.542
=====						
Omnibus:	61.380	Durbin-Watson:	1.744			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	165.171			
Skew:	1.168	Prob(JB):	1.36e-36			
Kurtosis:	6.432	Cond. No.	75.1			
=====						

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

가설 2. 고령자 비율이 높은 지역일수록 사고가 많을 것이다.

| 검정 방법

피어슨/스피어만 상관계수 + OLS 회귀

| 결과 요약

- ✓ 오히려 음의 상관관계
- ✓ 고령자 비율이 1%p 높을수록 사고 피해자 수 약 13명 감소
→ 인구밀도·교통량 등 외부 요인이 더 영향

```
# 매핑된 데이터프레임 생성
df_merged = pd.DataFrame(mapping)
# df_merged
HTML(df_merged.head(5).to_html(index=False))
```

시군구	고령자 비율 (%)	총 인구	고령자 수	사망자수	중상자수	경상자수	부상신고자수	피해자수
경상북도 의성군	44.38	50264	22279	16	57	31	1	105
전라남도 고흥군	43.30	61973	26816	9	84	46	0	139
경상남도 합천군	42.30	42102	17781	8	54	19	7	88
전라남도 보성군	41.16	38612	15870	10	37	36	2	85
경상북도 청도군	40.82	41533	16935	7	39	23	2	71

가설 3. 사고는 특정 시간대에 집중된다.

| 검정 방법

카이제곱 검정

| 결과 요약

✓ 사고는 특정 시간대(출퇴근 시간 등)에 몰림
→ 시간 기반 예측 모델 설계에 활용

```
chi2, p, dof, expected = chi2_contingency(contingency_df.T)
```

```
print(f"\n카이제곱 통계량: {chi2:.4f}")
```

```
print(f"p-value: {p:.10f}")
```

```
print(f"자유도: {dof}")
```

```
# =====  
# p-value는 0.0000이므로, 회귀계수가 유의미  
# 귀무가설 기각  
# 자유도 = 11 X 4 > 시간대는 12개, 연도는 5개  
# =====
```

카이제곱 통계량: 799.8412

p-value: 0.0000000000

자유도: 44

가설 3. 사고는 특정 시간대에 집중된다.

```
# 시간 데이터
df_time = pd.read_excel('../..data/preprocessed_data/시도_시군구_시간대별_노인교통사고_전처리ver.xlsx', header=[0, 1])
# df_time
HTML(df_time.head(5).to_html())
```

	시 도	시 군 구	기준 년도	2020	2021	2022	2023	2024	2020																	
	시 도	시 군 구	기준 년도	합계	합계	합계	합계	합계	0시 ~2 시	2시 ~4 시	4시 ~6 시	6시 ~8 시	8시 ~10 시	10 시 ~12 시	12 시 ~14 시	14 시 ~16 시	16 시 ~18 시	18 시 ~20 시	20 시 ~22 시	22 시 ~24 시	0시 ~2 시	2 시 ~4 시	4시 ~6 시	6시 ~8 시	8시 ~10 시	10 시 ~12 시
0	서울	합계	사고 [건]	5056	4829	5251	5731	5926	174	109	236	293	488	697	624	719	668	503	318	227	101	45	157	313	431	688
1	서울	합계	사망 [명]	86	103	87	71	107	1	5	14	6	6	7	7	16	8	8	3	5	3	1	7	17	5	15
2	서울	합계	부상 [명]	5279	5027	5543	6117	6309	188	109	239	304	507	732	661	756	694	524	327	238	104	44	155	316	453	720
3	서울	합계	(중 상자 [명])	1712	1627	1697	1816	1808	38	27	92	107	176	268	217	254	233	167	84	49	26	10	59	111	152	255
4	서울	종로구	사고 [건]	147	122	152	189	172	6	1	5	13	16	24	16	24	17	18	3	4	1	0	4	9	13	18

가설 4. 고령자 사고의 사망률은 비고령자보다 높다.

| 검정 방법

Z검정 + OLS 회귀

| 결과 요약

- ✓ 고령자 사망률 평균 3.01%
- ✓ 비고령자보다 평균 2.4%p 높음
- ✓ 고령자 여부만으로 사망률 70% 설명 가능

가설 4. 고령자 사고의 사망률은 비고령자보다 높다.

검정 방법

Z검정 + OLS 회귀

결과 요약

- 고령자 사망률 평균 3.01%
- 비고령자보다 평균 2.4%p 높음
- 고령자 여부만으로 사망률 70% 설명 가능

```
df_casualties = pd.read_excel('../data/preprocessed_data/연령층별_월별_사상자_전처리ver.xlsx', header=[0, 1])
# df_casualties
HTML(df_casualties.head(5).to_html())
```

✓ 0.1s

	연령	사고 년도	2020	2021	2022	2023	2024	2020																		
	연령	사고 년도	합계	합계	합계	합계	합계	01월	02월	03월	04월	05월	06월	07월	08월	09월	10월	11월	12월	01월	02월	03월	04월	05월	06월	07월
0	합계	사망 [명]	3081	2916	2735	2551	2521	277	222	239	213	254	254	240	295	301	309	261	216	197	203	209	212	255	227	254
1	합계	부상 [명]	306194	291608	281803	283799	278482	25442	23558	21823	23781	26791	27880	26969	26264	25839	28190	26698	22959	21323	21160	23928	25142	25125	25290	25870
2	합계	(중 상자 [명])	60564	55905	51715	52989	51553	5063	4463	4356	4844	5470	5614	5207	5123	5194	5554	5295	4381	4048	4040	4562	4983	4772	5019	4961
3	합계	(경 상자 [명])	226036	219283	212430	211569	207271	18696	17530	16155	17459	19563	20436	19933	19410	19001	20900	19832	17121	16141	15967	18022	18822	18911	18837	19459
4	12 세 이 하	사망 [명]	24	23	18	14	12	4	2	0	0	3	3	1	2	3	3	2	1	0	2	3	1	2	2	2

가설 4. 고령자 사고의 사망률은 비고령자보다 높다.

| 검정 방법

Z검정 + OLS 회귀

| 결과 요약

- ✓ 고령자 사망률 평균 3.01%
- ✓ 비고령자보다 평균 2.4%p 높음
- ✓ 고령자 여부만으로 사망률 70% 설명 가능

```
# 사망자 수와 사고 수 (각 집단의 성공/시도 수)
counts = [elderly_deaths, non_elderly_deaths]
nobs = [elderly_accidents, non_elderly_accidents]

# 이항 비율 검정 수행
z_stat, p_value = proportions_ztest(count=counts, nobs=nobs)
# 결과 출력
print(f"Z-통계량: {z_stat:.2f}")
print(f"p-값: {p_value:.10f}")

# =====
# 일반적으로  $|Z| > 1.96$ 이면 유의수준 0.05에서 유의하다고 판단
# Z-통계량이 108.47이므로 유의수준 0.05에서 유의하다고 판단
# p-value는 0.0000이므로, 회귀계수가 유의미
# 귀무가설 기각
# =====
```

Z-통계량: 108.47

p-값: 0.0000000000

가설 4. 고령자 사고의 사망률은 비고령자보다 높다.

검정 방법

Z검정 + OLS 회귀

결과 요약

- ✓ 고령자 사망률 평균 3.01%
- ✓ 비고령자보다 평균 2.4%p 높음
- ✓ 고령자 여부만으로 사망률 70% 설명 가능

```
# =====  
# const: 0.0060 > 비고령자의 평균 사망률: 0.6%  
# is_elderly: 0.0241 > 고령자일 경우 사망률이 평균 2.41% 증가  
# 고령자 사망률: 0.6 + 2.41 = 3.01%로 추정  
  
# R-squared: 0.704 > 고령자 여부 하나만으로 약 70.4%의 사망률 변동을 설명 가능  
# Adj. R-squared: 0.662 > 자유도 수정 후 66.2%  
  
# P > |t|(is_elderly)가 0.005이므로 통계적으로 유의미  
# 귀무가설 기각  
# =====
```

OLS Regression Results						
Dep. Variable:		사망률		R-squared:	0.704	
Model:		OLS		Adj. R-squared:	0.662	
Method:		Least Squares		F-statistic:	16.67	
Date:		Mon, 02 Jun 2025		Prob (F-statistic):	0.00467	
Time:		14:44:35		Log-Likelihood:	32.565	
No. Observations:		9		AIC:	-61.13	
Df Residuals:		7		BIC:	-60.74	
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	0.0060	0.003	2.152	0.068	-0.001	0.013
is_elderly	0.0241	0.006	4.083	0.005	0.010	0.038
Omnibus:	0.949	Durbin-Watson:		2.458		
Prob(Omnibus):	0.622	Jarque-Bera (JB):		0.009		
Skew:	0.076	Prob(JB):		0.995		
Kurtosis:	3.029	Cond. No.		2.55		

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

가설 5. 고령자 보행자 사고는 교차로/횡단보도에서 더 많이 발생한다.

| 검정 방법

카이제곱 검정 + 회귀 분석

| 결과 요약

- ✓ 약 62%가 교차로/횡단보도 주변에서 발생
- ✓ 유의미한 차이 있음, 그러나 설명력은 낮음($R^2=0.1$)

가설 5. 고령자 보행자 사고는 교차로/횡단보도에서 더 많이 발생한다.

| 검정 방법

카이제곱 검정 + 회귀 분석

| 결과 요약

- ✓ 약 62%가 교차로/횡단보도 주변에서 발생
- ✓ 유의미한 차이 있음, 그러나 설명력은 낮음($R^2=0.1$)

```
# 카이제곱 검정 수행
chi2, p, dof, expected = chi2_contingency(contingency_table)

print(f"카이제곱 값: {chi2:.2f}")
print(f"p-value: {p:.15f}")

# =====
# p-value는 0.0000이므로, 회귀계수가 유의미
# 귀무가설 기각
# =====
```

```
카이제곱 값: 5163.84
p-value: 0.000000000000000
```

가설 5. 고령자 보행자 사고는 교차로/횡단보도에서 더 많이 발생한다.

검정 방법

카이제곱 검정 + 회귀 분석

결과 요약

- ✓ 약 62%가 교차로/횡단보도 주변에서 발생
- ✓ 유의미한 차이 있음, 그러나 설명력은 낮음(R²=0.1)

```
# =====  
# Intercept (기준: 차대사람 - 기타):      0.2906      >>>   '기타' 사고유형은 교차로/횡단보도에서 발생할 확률이 약 29.1%  
# 차대사람 - 길가장자리구역통행중:      -0.0267      >>>   기준보다 2.7%p 낮음, 더 기타도로에서 발생함  
# 차대사람 - 보도통행중:                  0.0396      >>>   기준보다 4.0%p 높음, 교차로/횡단보도에서 더 자주 발생  
# 차대사람 - 차도통행중:                  0.1016      >>>   기준보다 10.2%p 높음, 교차로/횡단보도 사고 비중 큼  
# 차대사람 - 횡단중:                      0.3375      >>>   기준보다 33.8%p 더 높음, 명확히 교차로 관련  
  
# 모든 변수의 p-value < 0.01이므로 모든 변수가 통계적 유의미  
  
# R-squared = 0.100 > 사고유형만으로는 전체 위치 분포의 10% 정도 설명 가능  
# 즉, 해당 사고가 교차로/횡단보도에서 발생했는지 아닌지를 정확히 예측하기엔 부족  
  
# 귀무가설 기각  
# =====
```

OLS Regression Results							
Dep. Variable:	위치_이진	R-squared:	0.100				
Model:	OLS	Adj. R-squared:	0.100				
Method:	Least Squares	F-statistic:	1434.				
Date:	Mon, 02 Jun 2025	Prob (F-statistic):	0.00				
Time:	14:44:04	Log-Likelihood:	-34293.				
No. Observations:	51671	AIC:	6.860e+04				
Df Residuals:	51666	BIC:	6.864e+04				
Df Model:	4						
Covariance Type:	nonrobust						
		coef	std err	t	P> t	[0.025	0.975]
	const	0.2906	0.003	87.629	0.000	0.284	0.297
사고유형_차대사람___길가장자리구역통행중		-0.0267	0.010	-2.791	0.005	-0.045	-0.008
사고유형_차대사람___보도통행중		0.0396	0.010	4.124	0.000	0.021	0.058
사고유형_차대사람___차도통행중		0.1016	0.007	15.276	0.000	0.089	0.115
사고유형_차대사람___횡단중		0.3375	0.005	71.412	0.000	0.328	0.347
Omnibus:	325371.444	Durbin-Watson:	1.949				
Prob(Omnibus):	0.000	Jarque-Bera (JB):	5304.663				
Skew:	0.255	Prob(JB):	0.00				
Kurtosis:	1.515	Cond. No.	5.64				

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
df_total = pd.read_csv('../data/preprocessed_data/시도_시군구별_보행자_사고_사고유형_전처리ver.csv')  
# df_total.head()  
HTML(df_total.head().to_html())
```

	연도	시군구	법정동코드	사고내용	사망자수	중상자수	경상자수	부상신고자수	사고유형	도로형태	피해자 상해정도
0	2020	서울특별시 종로구 창성동	1111010500	중상사고	0	1	0	0	차대사람 - 차도통행중	교차로 - 교차로부근	중상
1	2020	서울특별시 종로구 창성동	1111010500	중상사고	0	1	0	0	차대사람 - 차도통행중	교차로 - 교차로안	중상
2	2020	서울특별시 종로구 통인동	1111010800	경상사고	0	0	1	0	차대사람 - 기타	단일로 - 기타	경상
3	2020	서울특별시 종로구 누상동	1111010900	경상사고	0	0	1	0	차대사람 - 차도통행중	교차로 - 교차로안	경상
4	2020	서울특별시 종로구 누상동	1111010900	경상사고	0	0	1	0	차대사람 - 기타	단일로 - 기타	경상



3. 분석 결과



3가지 모델 소개



PredictAccidentOccur

.....

특정 시간대와 지역 정보를
기반으로 사고 발생 여부 예측



HighRiskRank

.....

사고 유형, 도로 형태, 피해 규모
기반 위험 점수 산정



PredictWithTime

.....

포아송 회귀 모델 기반
지역 + 시간대 → 사고 건수 예측



PredictAccidentOccur

특정 시간대와 지역 정보를 기반으로 사고 발생 여부 예측

```
Verbosity: 2 (Standard Logging)
===== System Info =====
AutoGluon Version: 1.3.1
Python Version: 3.11.11
Operating System: Darwin
Platform Machine: arm64
Platform Version: Darwin Kernel Version 24.4.0: Fri Apr 11 18:33:39 PDT 2025; root:xnu-11417.101.15~117/RELEASE_ARM64_T6020
CPU Count: 12
Memory Avail: 2.91 GB / 16.00 GB (18.2%)
Disk Space Avail: 24.80 GB / 460.43 GB (5.4%)
=====
No presets specified! To achieve strong results with AutoGluon, it is recommended to use the available presets. Defaulting to `medium`...
Recommended Presets (For more details refer to https://auto.gluon.ai/stable/tutorials/tabular/tabular-essentials.html#presets):
presets='experimental' : New in v1.2: Pre-trained foundation model + parallel fits. The absolute best accuracy without consideration for inference speed. Does not support GPU.
presets='best' : Maximize accuracy. Recommended for most users. Use in competitions and benchmarks.
presets='high' : Strong accuracy with fast inference speed.
presets='good' : Good accuracy with very fast inference speed.
presets='medium' : Fast training time, ideal for initial prototyping.
Beginning AutoGluon training ...
AutoGluon will save models to "/Users/leehyunseung/Documents/GitHub/1_team_project/notebooks/03_modeling/PredictAccidentOccur"
Train Data Rows: 21216
Train Data Columns: 4
Label Column: 사고여부
AutoGluon infers your prediction problem is: 'binary' (because only two unique label-values observed).
...
AutoGluon training complete, total runtime = 19.95s ... Best model: WeightedEnsemble_L2 | Estimated inference throughput: 54439.5 rows/s (2122 batch size)
Disabling decision threshold calibration for metric `accuracy` due to having fewer than 10000 rows of validation data for calibration, to avoid overfitting (2122 rows).
`accuracy` is generally not improved through threshold calibration. Force calibration via specifying `calibrate_decision_threshold=True`.
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/Users/leehyunseung/Documents/GitHub/1_team_project/notebooks/03_modeling/PredictAccidentOccur")
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

{'accuracy': 0.9958521870286576,
 'balanced_accuracy': 0.9958521870286576,
 'mcc': 0.9917325763042215,
 'roc_auc': 0.9986205237403002,
 'f1': 0.9958677685950413,
 'precision': 0.9921407185628742,
 'recall': 0.9996229260935143}
```

| 방식

Negative Sampling을 활용한 이진 분류

| 알고리즘

AutoGluon – WeightedEnsemble_L2

| 성능

정확도: 99.6%
예측 시간: 0.038초

| 활용

사고 발생 가능 지역에 사전 사고 발생 여부 제공



PredictAccidentOccur

.....
특정 시간대와 지역 정보를 기반으로 사고 발생 여부 예측

	model	score_val	eval_metric	pred_time_val	fit_time	\
0	WeightedEnsemble_L2	0.996230	accuracy	0.038979	1.477922	
1	CatBoost	0.995759	accuracy	0.001273	0.908944	
2	NeuralNetFastAI	0.995759	accuracy	0.011406	8.003872	
3	LightGBMXT	0.994345	accuracy	0.010126	1.925405	
4	LightGBM	0.993402	accuracy	0.006112	1.115939	
5	LightGBMLarge	0.992460	accuracy	0.010680	2.229755	
6	RandomForestGini	0.875118	accuracy	0.036283	0.478133	
7	RandomForestEntr	0.863808	accuracy	0.037409	0.528932	
8	XGBoost	0.860509	accuracy	0.009552	1.625571	
9	ExtraTreesGini	0.784637	accuracy	0.037150	0.396594	
10	ExtraTreesEntr	0.781810	accuracy	0.037565	0.414714	
	pred_time_val_marginal	fit_time_marginal	stack_level	can_infer	\	
0	0.000297	0.040046	2	True		
1	0.001273	0.908944	1	True		
2	0.011406	8.003872	1	True		
3	0.010126	1.925405	1	True		
4	0.006112	1.115939	1	True		
5	0.010680	2.229755	1	True		
6	0.036283	0.478133	1	True		
7	0.037409	0.528932	1	True		
8	0.009552	1.625571	1	True		
9	0.037150	0.396594	1	True		
10	0.037565	0.414714	1	True		
...						
7	4					
8	9					
9	6					
10	7					

| 방식

Negative Sampling을 활용한 이진 분류

| 알고리즘

AutoGluon – WeightedEnsemble_L2

| 성능

정확도: 99.6%

예측 시간: 0.038초

| 활용

사고 발생 가능 지역에 사전 사고 발생 여부 제공



HighRiskRank

사고 유형, 도로 형태, 피해 규모 기반 위험 점수 산정

```
(_ray_fit pid=19886) [1000] valid_set's multi_error: 0.3231
(raylet) [2025-06-05 16:22:30,331 E 19328 29509675] (raylet) file_system_monitor.cc:116: /tmp/ray/session 2025-06-05 16-18-57 964745 19122 is over 95% full, available space:
(_ray_fit pid=19815) Ran out of time, early stopping on iteration 1415. Best iteration is:
(_ray_fit pid=19815) [1403] valid_set's multi_error: 0.309819
(raylet) [2025-06-05 16:22:40,440 E 19328 29509675] (raylet) file_system_monitor.cc:116: /tmp/ray/session 2025-06-05 16-18-57 964745 19122 is over 95% full, available space:
(_ray_fit pid=19888) Ran out of time, early stopping on iteration 1436. Best iteration is: [repeated 7x across cluster]
(_ray_fit pid=19888) [1434] valid_set's multi_error: 0.306838 [repeated 7x across cluster]
(raylet) [2025-06-05 16:22:50,495 E 19328 29509675] (raylet) file_system_monitor.cc:116: /tmp/ray/session 2025-06-05 16-18-57 964745 19122 is over 95% full, available space:
(_dystack pid=19341) 0.6951 = Validation score (accuracy)
(_dystack pid=19341) 38.12s = Training runtime
(_dystack pid=19341) 91.46s = Validation runtime
(_dystack pid=19341) Fitting model: WeightedEnsemble_L3 ... Training model for up to 220.72s of the -8.47s of remaining time.
(_dystack pid=19341) Ensemble Weights: {'LightGBMXT_BAG_L2': 1.0}
(_dystack pid=19341) 0.6951 = Validation score (accuracy)
(_dystack pid=19341) 1.06s = Training runtime
(_dystack pid=19341) 0.0s = Validation runtime
(_dystack pid=19341) AutoGluon training complete, total runtime = 232.29s ... Best model: WeightedEnsemble_L3 | Estimated inference throughput: 48.5 rows/s (4593 batch size)
(_dystack pid=19341) Automatically performing refit_full as a post-fit operation (due to `.fit(..., refit_full=True)`)
(_dystack pid=19341) Refitting models via `predictor.refit_full` using all of the data (combined train and validation)...
(_dystack pid=19341) Models trained in this way will have the suffix "_FULL" and have NaN validation score.
(_dystack pid=19341) This process is not bound by time_limit, but should take less time than the original `predictor.fit` call.
(_dystack pid=19341) To learn more, refer to the `.refit_full` method docstring which explains how "_FULL" models differ from normal models.
(_dystack pid=19341) Fitting model: KNeighborsUnif_BAG_L1_FULL | Skipping fit via cloning parent ...
(_dystack pid=19341) 0.03s = Training runtime
(_dystack pid=19341) 0.96s = Validation runtime
(_dystack pid=19341) Fitting model: KNeighborsDist_BAG_L1_FULL | Skipping fit via cloning parent ...
...
Updated best model to "CatBoost_BAG_L1_FULL" (Previously "WeightedEnsemble_L2"). AutoGluon will default to using "CatBoost_BAG_L1_FULL" for predict() and predict_proba().
Refit complete, total runtime = 91.37s ... Best model: "CatBoost_BAG_L1_FULL"
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/Users/leehyunseung/Documents/GitHub/1_team_project/notebooks/03_modeling/HighRiskRank")
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/Users/leehyunseung/Documents/GitHub/1_team_project/notebooks/03_modeling/HighRiskRank")

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
{'accuracy': 0.595355587808418,
 'balanced_accuracy': 0.5860691577804529,
 'mcc': 0.49793465678837673}
```

| 방식

사고 유형, 도로 형태, 피해 규모를 고려한 다중분류

| 알고리즘

AutoGluon – CatBoost_BAG_L1

| 성능

정확도: 59.8%
위험도 4 재현율: 약 90% | 위험도 5 정밀도: 약 72%

| 활용

고위험 지역을 한눈에 파악할 수 있도록 시각화 제공



HighRiskRank

.....
사고 유형, 도로 형태, 피해 규모 기반 위험 점수 산정

	model	score_val	eval_metric	pred_time_val \
0	CatBoost_BAG_L1	0.598026	accuracy	0.289819
1	WeightedEnsemble_L2	0.598026	accuracy	0.292404
2	CatBoost_r177_BAG_L1	0.597711	accuracy	0.378128
3	CatBoost_r9_BAG_L1	0.591954	accuracy	0.219294
4	XGBoost_BAG_L1	0.584720	accuracy	0.788037
5	LightGBMXT_BAG_L1	0.583245	accuracy	0.813826
6	LightGBM_r96_BAG_L1	0.582277	accuracy	2.806550
7	LightGBM_r131_BAG_L1	0.580390	accuracy	1.115063
8	LightGBM_BAG_L1	0.579229	accuracy	0.256374
9	NeuralNetFastAI_BAG_L1	0.576350	accuracy	0.351416
10	NeuralNetFastAI_r191_BAG_L1	0.575576	accuracy	0.654792
11	LightGBMLarge_BAG_L1	0.575358	accuracy	0.451282
12	RandomForestGini_BAG_L1	0.519329	accuracy	1.566191
13	RandomForestEntr_BAG_L1	0.517902	accuracy	1.292379
14	ExtraTreesGini_BAG_L1	0.514297	accuracy	1.092383
15	ExtraTreesEntr_BAG_L1	0.513281	accuracy	1.040090
16	KNeighborsUnif_BAG_L1	0.242960	accuracy	1.487708
17	KNeighborsDist_BAG_L1	0.240105	accuracy	1.422632
18	ExtraTreesEntr_BAG_L1_FULLL	NaN	accuracy	1.040090
19	ExtraTreesGini_BAG_L1_FULLL	NaN	accuracy	1.092383
20	RandomForestEntr_BAG_L1_FULLL	NaN	accuracy	1.292379
21	KNeighborsDist_BAG_L1_FULLL	NaN	accuracy	1.422632
22	KNeighborsUnif_BAG_L1_FULLL	NaN	accuracy	1.487708
23	RandomForestGini_BAG_L1_FULLL	NaN	accuracy	1.566191
...				
32	True	30		
33	True	34		
34	True	31		
35	True	26		

| 방식

사고 유형, 도로 형태, 피해 규모를 고려한 다중분류

| 알고리즘

AutoGluon – CatBoost_BAG_L1

| 성능

정확도: 59.8%
위험도 4 재현율: 약 90% | 위험도 5 정밀도: 약 72%

| 활용

고위험 지역을 한눈에 파악할 수 있도록 시각화 제공



HighRiskRank

.....
사고 유형, 도로 형태, 피해 규모 기반 위험 점수 산정

```
from sklearn.metrics import classification_report

# 테스트 데이터 예측
y_true = test_data[target_col]
y_pred = predictor.predict(test_data)

print(classification_report(y_true, y_pred, digits=3))
```

	precision	recall	f1-score	support
1	0.555	0.554	0.554	2127
2	0.638	0.207	0.313	2008
3	0.557	0.615	0.584	2257
4	0.590	0.895	0.711	2427
5	0.719	0.660	0.688	1516
accuracy			0.595	10335
macro avg	0.612	0.586	0.570	10335
weighted avg	0.604	0.595	0.570	10335

| 방식

사고 유형, 도로 형태, 피해 규모를 고려한 다중분류

| 알고리즘

AutoGluon – CatBoost_BAG_L1

| 성능

정확도: 59.8%

위험도 4 재현율: 약 90% | 위험도 5 정밀도: 약 72%

| 활용

고위험 지역을 한눈에 파악할 수 있도록 시각화 제공



PredictWithTime

.....
포아송 회귀 모델 기반 | 지역 + 시간대 → 사고 건수 예측

```
df_eval = df_model.copy()

# 예측값을 반올림하여 정수로 변환
df_eval['예측_이진'] = (df_eval['예측_사고건수'].round() >= 1).astype(int)
df_eval['실제_이진'] = (df_eval['사고건수'] >= 1).astype(int)

# 평가 지표 계산
precision = precision_score(df_eval['실제_이진'], df_eval['예측_이진'], zero_division=0)
recall = recall_score(df_eval['실제_이진'], df_eval['예측_이진'], zero_division=0)
accuracy = accuracy_score(df_eval['실제_이진'], df_eval['예측_이진'])

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"Accuracy: {accuracy:.4f}")
```

```
Precision: 0.8875
Recall: 1.0000
Accuracy: 0.8875
```

| 모델

포아송 회귀 (Poisson Regression)

| 예측 성능 지표

Accuracy: 88.7%

Recall: 1.000

RMS (Root Mean Square Error): ±11건

MAE (Mean Absolute Error): 약 7건

Cosine Similarity: 0.8313

상관계수: 0.6717 (양의 상관관계)

| 활용

특정 시간대의 사고규모를 사전 예측하여 대응 가능
정책적 수요 예측 및 인력 배치 최적화 기여



PredictWithTime

.....
포아송 회귀 모델 기반 | 지역 + 시간대 → 사고 건수 예측

===== 결론 =====

RMS: 11.287 - 평균적으로 약 ±11건 오차
MAE: 7.138 - 평균 절대 오차는 약 7건 수준
Cosine Similarity: 0.8313 - 예측값과 실제값의 패턴이 83% 이상 유사
Correlation Coefficient: 0.6717 - 예측값과 실제값 사이에 양의 상관관계 있음

RMSE: 11.2872
MAE: 7.1376
Cosine Similarity: 0.8313
Correlation Coefficient: 0.6717

| 모델

포아송 회귀 (Poisson Regression)

| 예측 성능 지표

Accuracy: 88.7%
Recall: 1.000
RMS (Root Mean Square Error): ±11건
MAE (Mean Absolute Error): 약 7건
Cosine Similarity: 0.8313
상관계수: 0.6717 (양의 상관관계)

| 활용

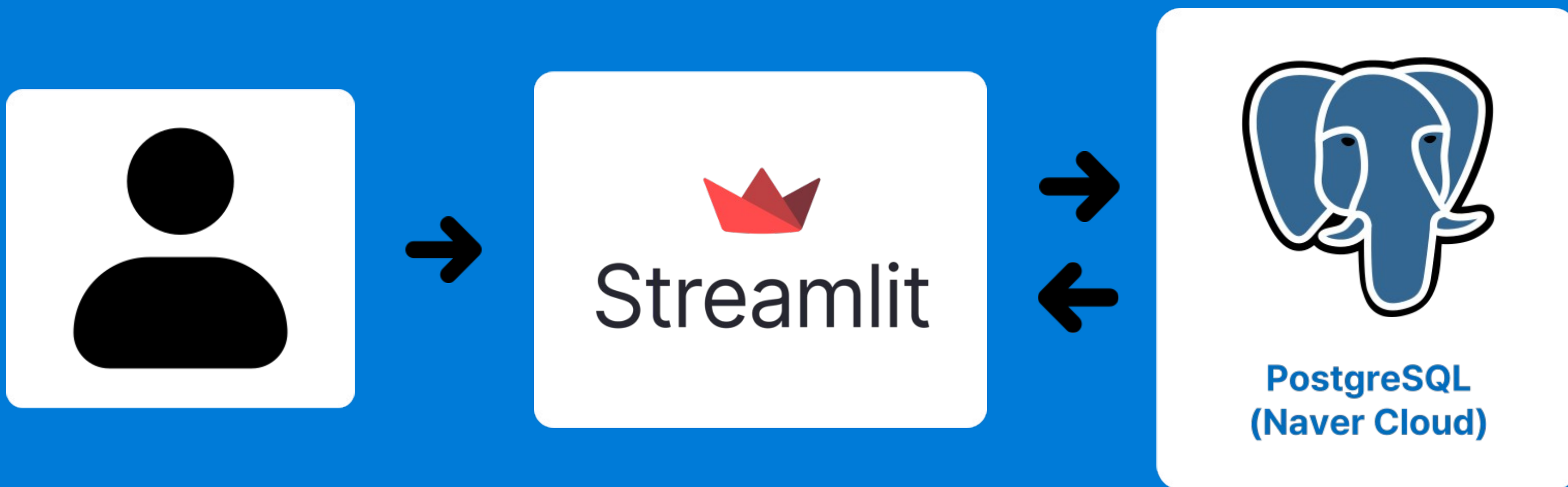
특정 시간대의 사고규모를 사전 예측하여 대응 가능
정책적 수요 예측 및 인력 배치 최적화 기여



4. 서비스 구현



서비스 아키텍처 및 배포 구조



사용 기술 스택

FrontEnd



DataBase



PostgreSQL
(Naver Cloud)

Map



Folium

Data / Backend



배포

NAVER
Cloud



2025.06.09

▣ 직관이 아닌 데이터로 정책을 설계하다.

Data over intuition.

주요 메인 화면



2025.06.09

▣ 직관이 아닌 데이터로 정책을 설계하다.

Data over intuition.

이외 화면



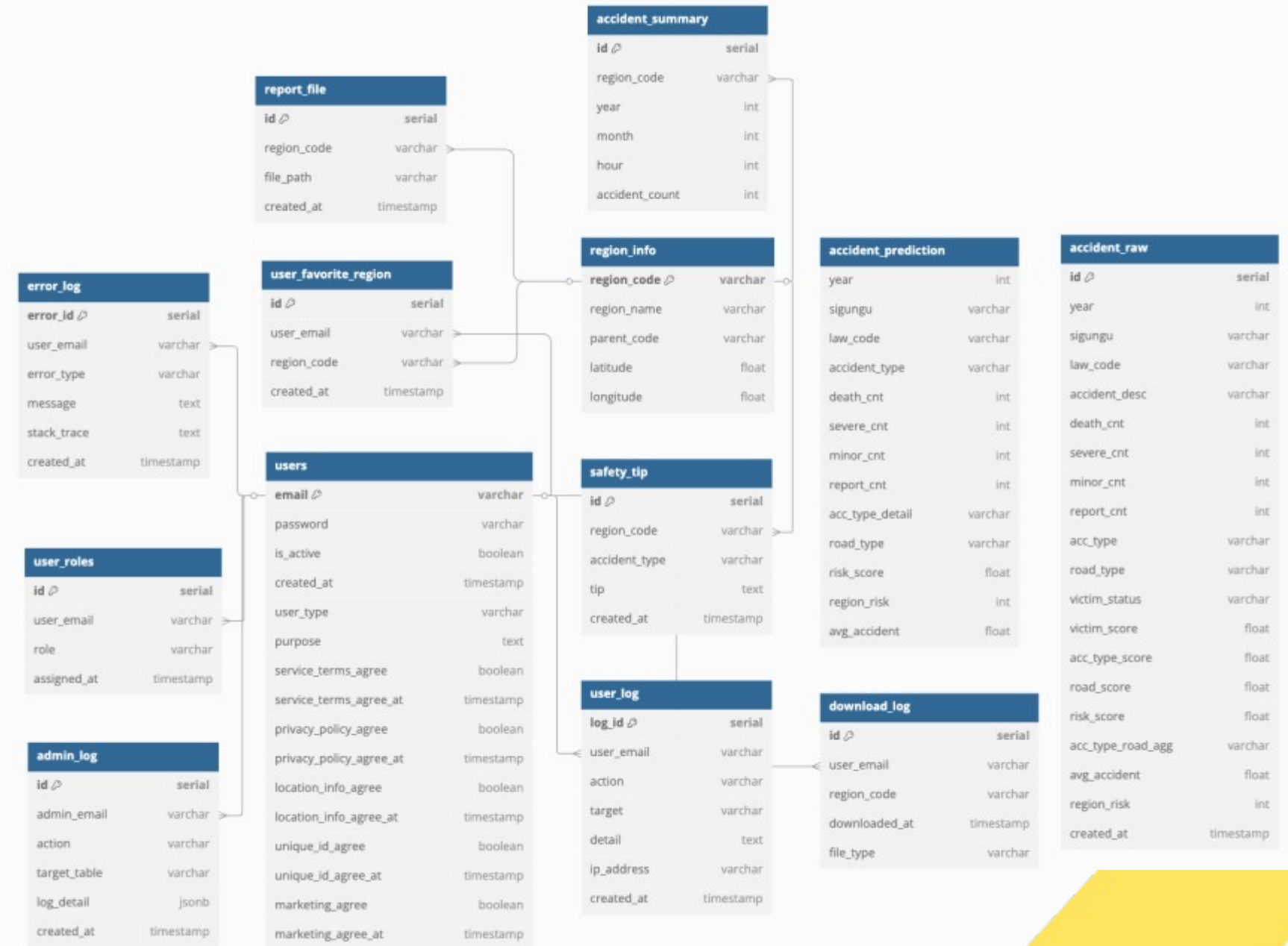
DB 테이블 설계 및 관계(ERD)

| 주요 구성

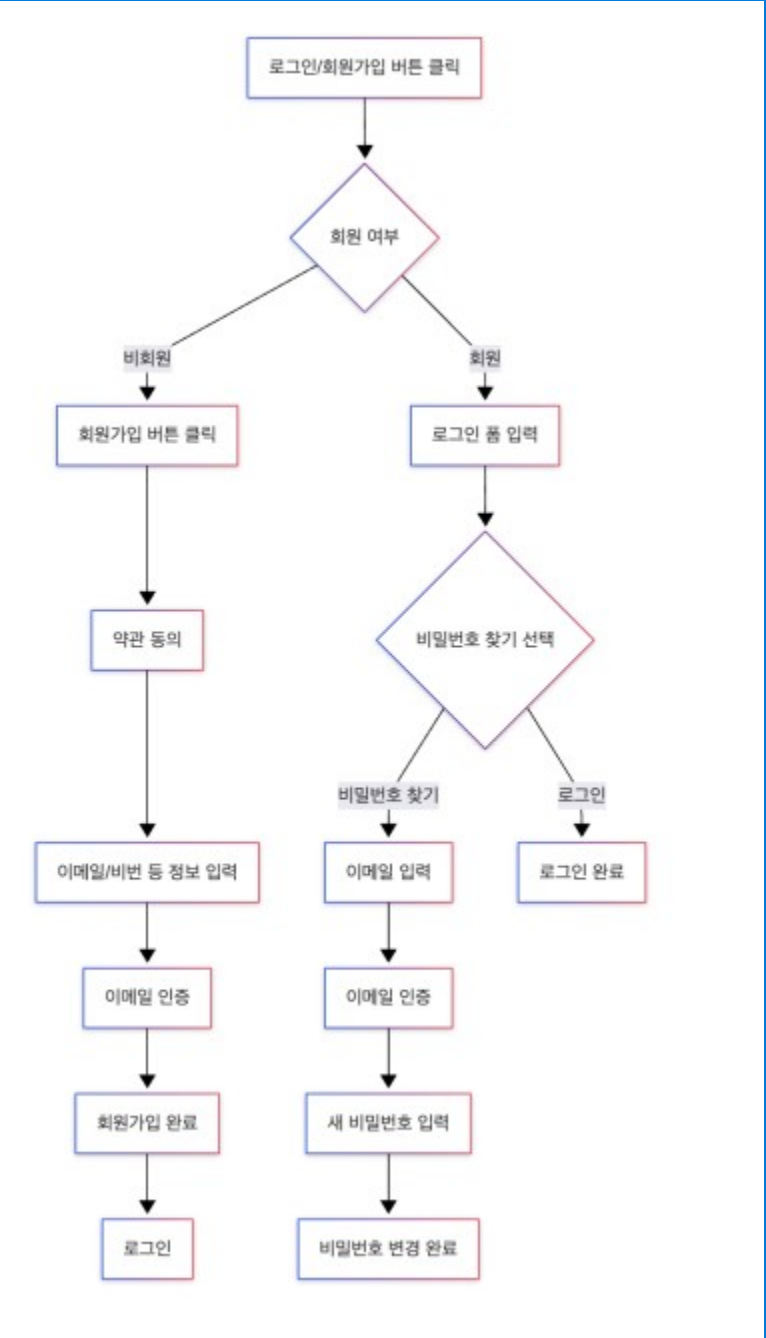
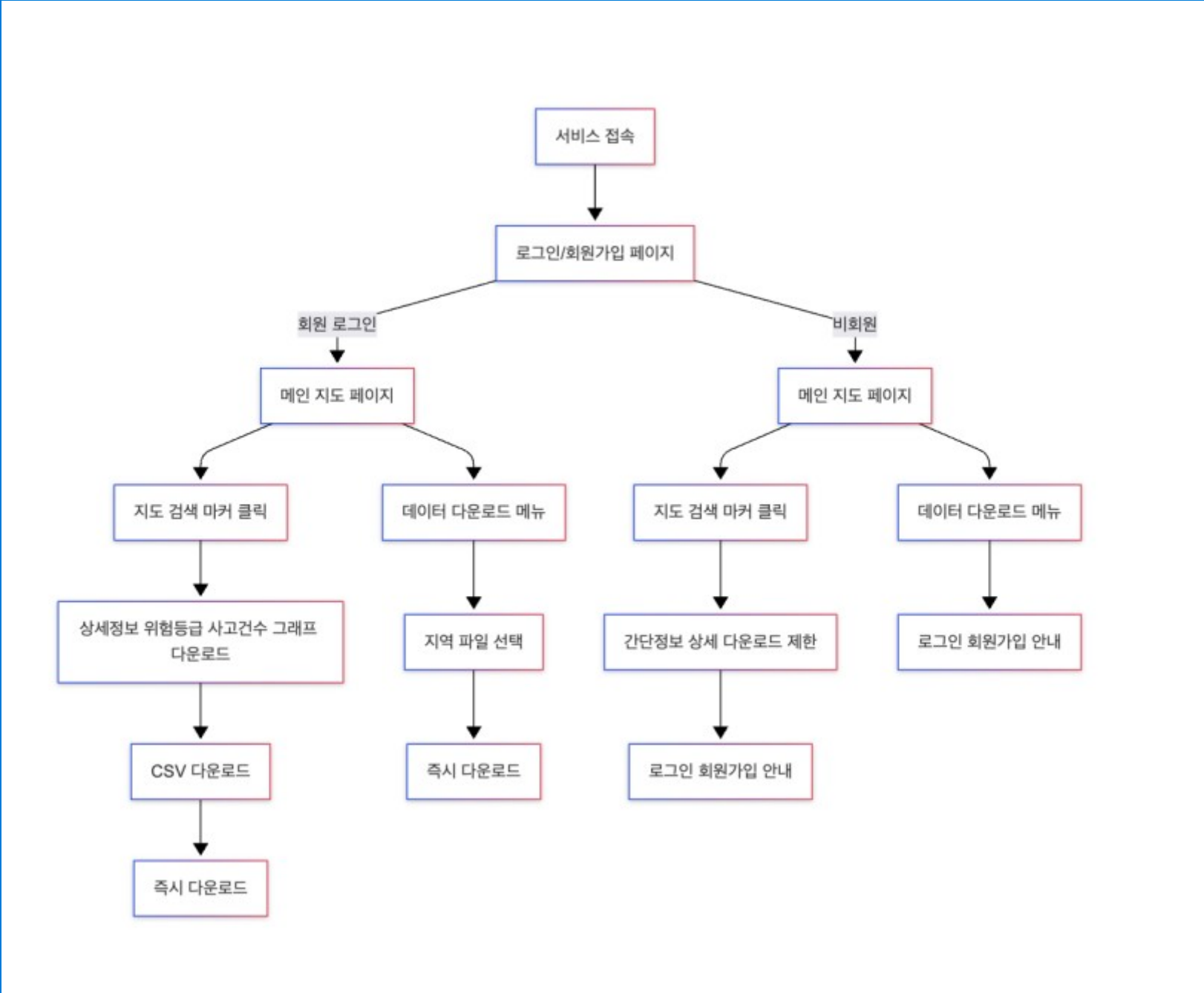
- 회원 관리: 사용자 정보, 역할, 약관 동의 등
- 지역 정보: 구별 행정구역 및 좌표
- 사고 데이터: 원본(raw) 데이터 | 사고 예측 결과(모델 기반) | 월/시간별 사고 통계
- 사용자 활동: 즐겨찾기 지역 | 다운로드 기록 | 사용자 로그 | 에러 기록 등
- 관리자 기능: 리포트 파일 관리 | 관리자 행위 로그

| 특징

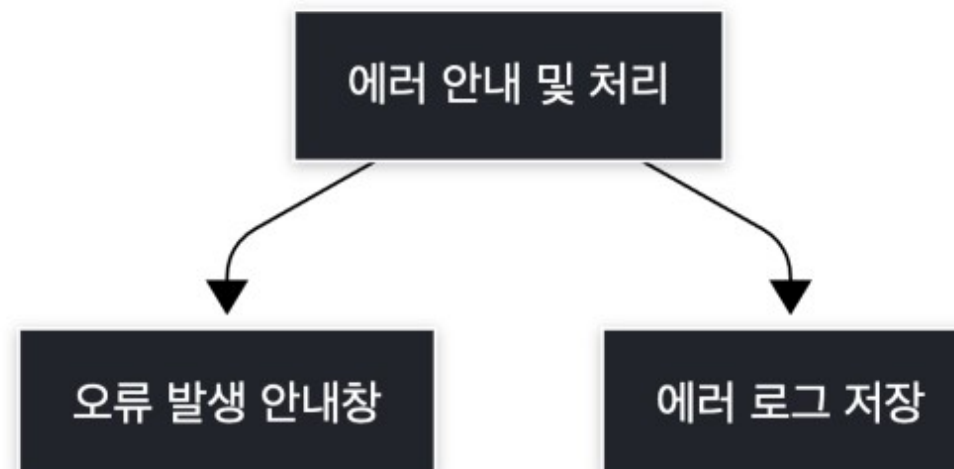
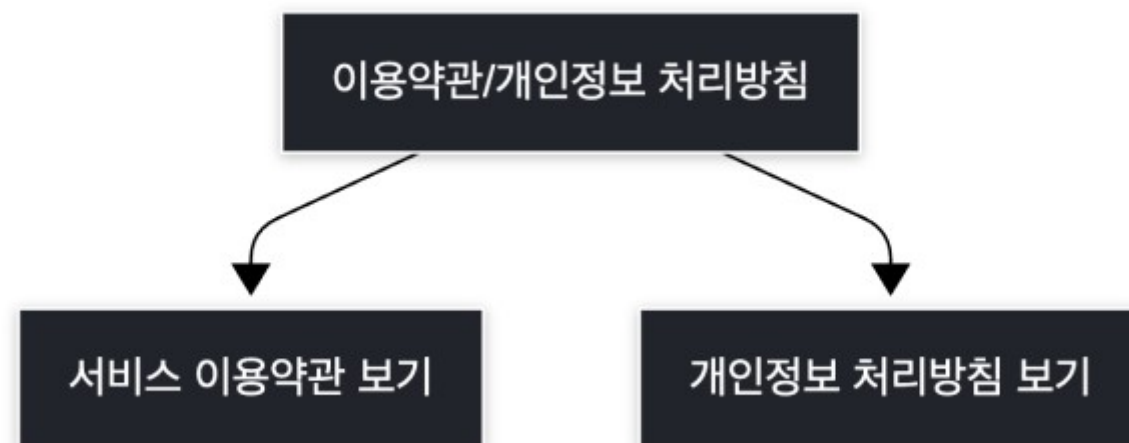
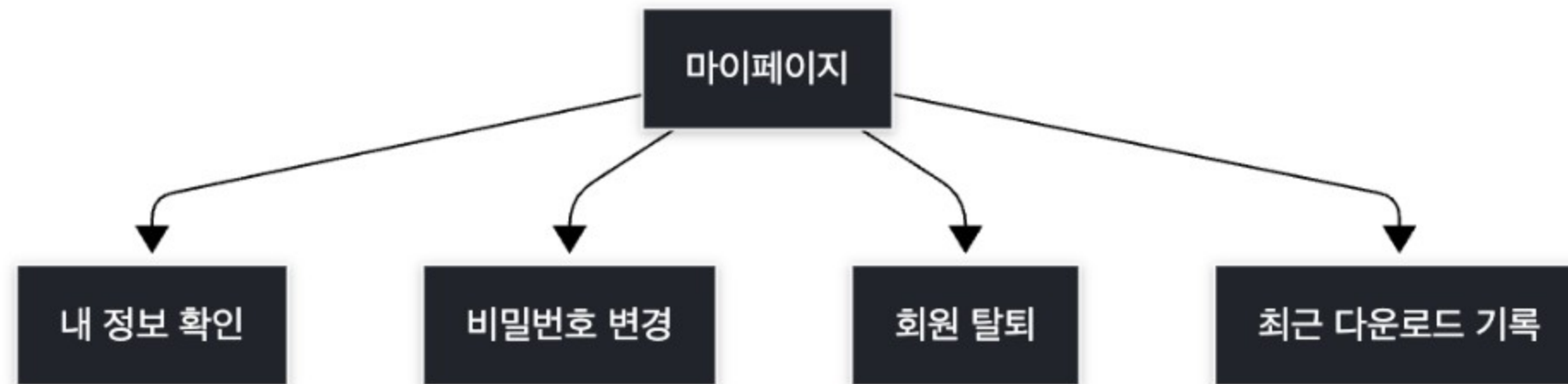
- ✓ 테이블 간 외래키(FK) 관계로 데이터 무결성 보장
- ✓ 원본 데이터와 가공 데이터 분리 저장
- ✓ 사용자 중심 서비스 및 통계 제공



주요 기능 서비스 플로우 차트



부가 기능 서비스 플로우 차트



운영/보안 / 관리

이력/로그 관리

모든 사용자와 관리자의 로그인, 데이터 다운로드, 정보 조회 등 주요 행위에 대한 이력과 로그를 남기고, 보안 및 감사 목적의 모니터링 체계를 갖추었습니다.

DB 백업 및 복구

정기적으로 데이터베이스 전체 백업을 수행하며, 데이터 손실이나 장애 발생 시 신속하게 복구할 수 있도록 백업 데이터를 별도 저장소에 안전하게 관리합니다.

암호화 적용

사용자 비밀번호 등 민감한 정보는 해시 함수를 이용해 암호화하여 저장하며, 원본 데이터에 접근이 불가하도록 안전하게 관리합니다.

기본 보안 적용

SQL Injection 등 주요 웹 보안 취약점에 대응할 수 있도록 입력값 검증, 파라미터 바인딩 등 보안 코딩을 적용하고, 최소 권한 원칙을 따르는 등 안전한 서비스 환경을 구축했습니다.





5. 활용성



활용 및 확장성

정책 수립 및 예산 편성 지원

데이터 기반으로 위험 지역을 선정해 예산 편성과 정책 결정에 활용
→ 예: 고위험 지역에 우선적으로 안전 시설물을 설치하거나, 맞춤형 교통안전 대책 수립

실무자 신속한 의사결정

사고 다발 구간, 시간대 파악 → 경찰/공무원의 순찰·단속 시간대 조정
예: 밤 시간대 고령자 사고 집중구간에 야간 순찰 강화

일반 시민에게 정보 제공

지역별 사고 위험 시각화 → 누구나 내 주변 사고 위험도 한눈에 확인
예: 실시간 지도에서 우리 동네 위험도, 사고 유형 즉시 확인

서비스 확장성

데이터/분석모델 재활용 → 다른 연령층, 전국/도시 확장 서비스 개발
예: 어린이·장애인 등 교통약자 대상 위험 분석 추가

2025.06.09

▣ 직관이 아닌 데이터로 정책을 설계하다.

Data over intuition.



Q & A

